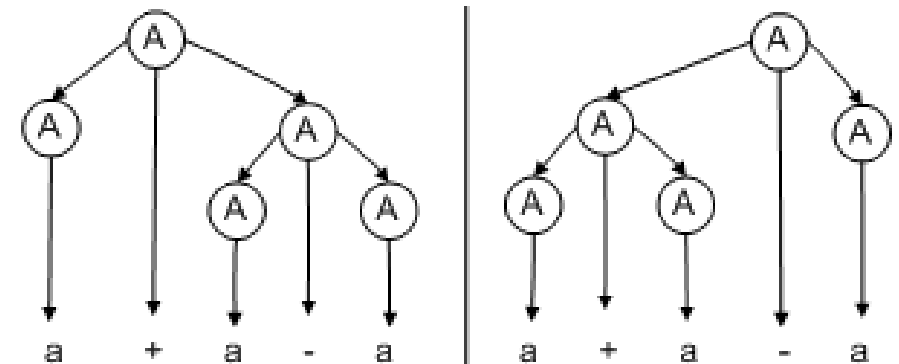


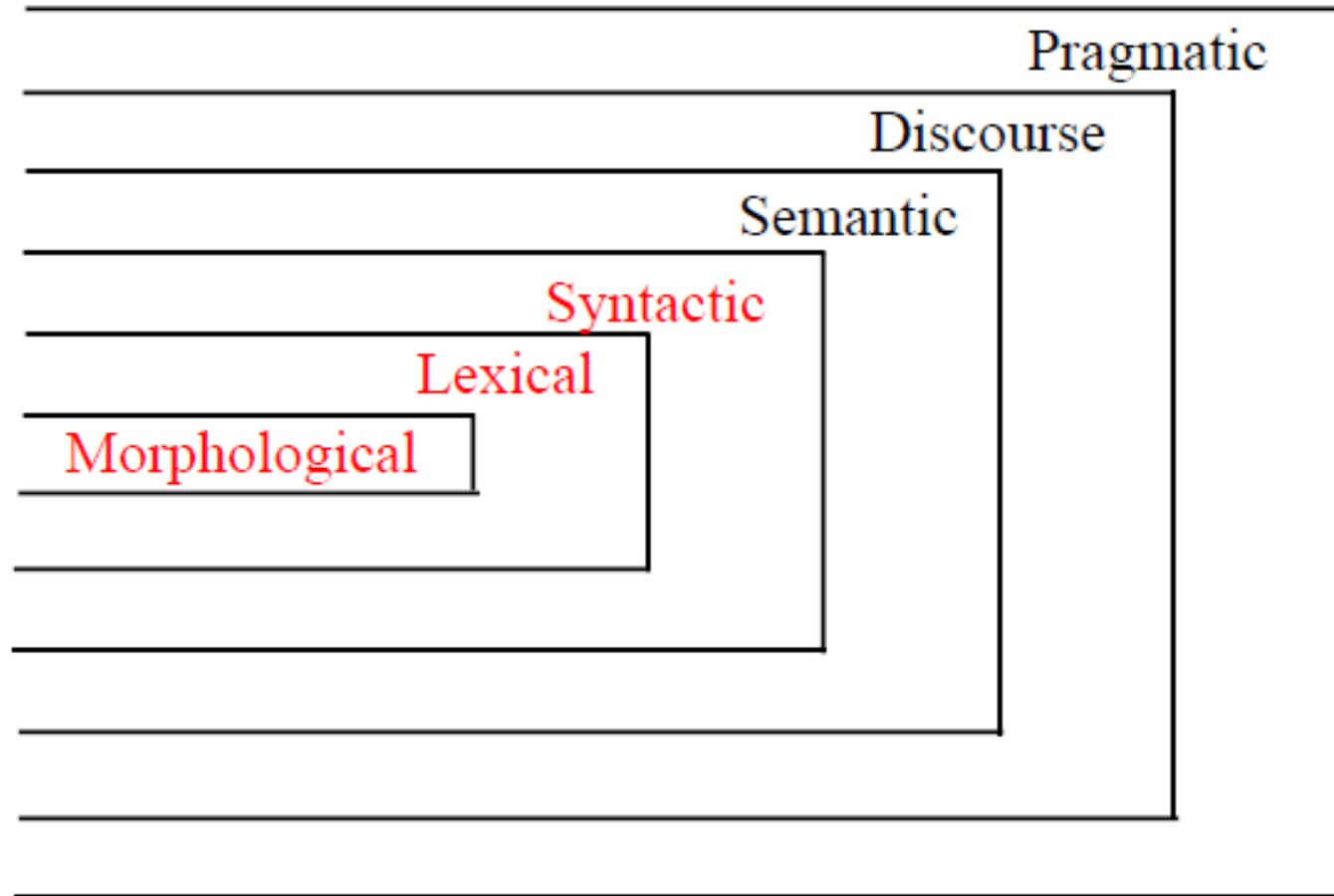
Parsing Algorithms

(Natural Language Processing)

Randil Pushpananda
rpn@ucsc.cmb.ac.lk



Synchronic Model of Language

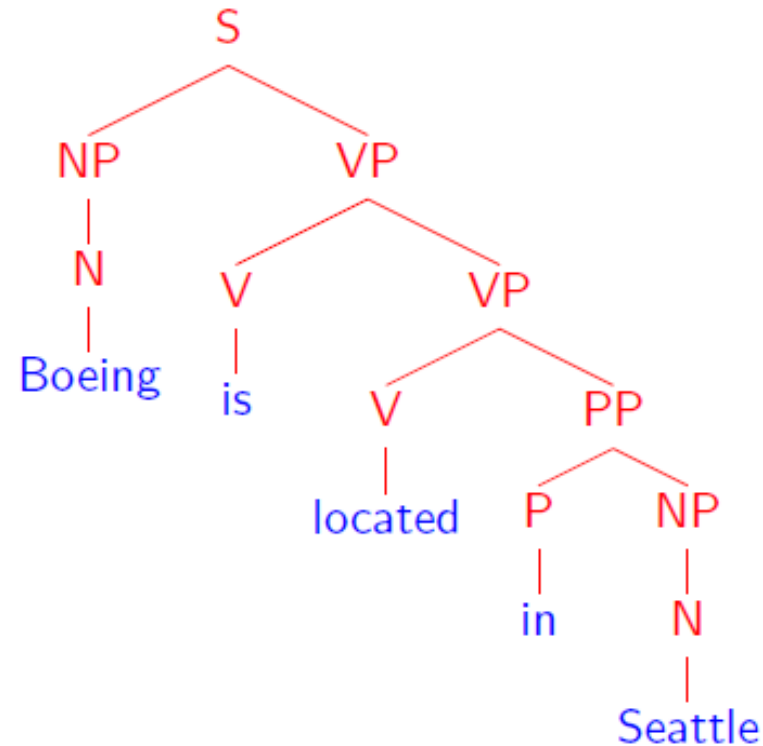


Parsing (Syntactic Structure)

INPUT:

Boeing is located in Seattle.

OUTPUT:



Parsing defined:

- The process of finding a derivation (i. e. sequence of productions) leading from the START symbol to a TERMINAL symbol
 - Shows how a particular sentence could be generated by the rules of the grammar
- If sentence is **structurally ambiguous**, more than one possible derivation is produced
- Can solve both the recognition and analysis problems
 - Is this sentence derived from this grammar?
 - Give the derivation(s) that can derive this sentence.
- Parsing algorithms give a strategy for finding a derivation by making choices among the derivation rules and deciding when the derivation is complete or not.

Top-Down Parser

- Hypothesis-driven
 - At each stage, parser hypothesizes a structure, and tests whether data (next word in sentence) fits the hypothesis
- Looks at goal first (S) and then sees which rules can be applied
 - Typically progresses from top-to-bottom, left-to-right
 - Non-deterministic (can be rewritten in more than one way)
- When rules derive lexical elements (words), check with the input to see if the right sentence is being derived
- An algorithm may include a backtracking mechanism
 - When it is determined that the wrong rule has been used, it backs up and tries another rule

Example Grammar

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Pronoun$

$NP \rightarrow Proper-Noun$

$NP \rightarrow Det Nominal$

$Nominal \rightarrow Noun$

$Nominal \rightarrow Nominal Noun$

$Nominal \rightarrow Nominal PP$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$VP \rightarrow Verb NP PP$

$VP \rightarrow Verb PP$

$VP \rightarrow VP PP$

$PP \rightarrow Preposition NP$

$Det \rightarrow that \mid this \mid a$

$Noun \rightarrow book \mid ight \mid meal \mid money$

$Verb \rightarrow book \mid include \mid prefer$

$Pronoun \rightarrow I \mid she \mid me$

$Proper-Noun \rightarrow Houston \mid TWA$

$Aux \rightarrow does$

$Preposition \rightarrow from \mid to \mid on \mid near \mid through$

Example Derivation

- Derivation for “Book that flight” (from the grammar)

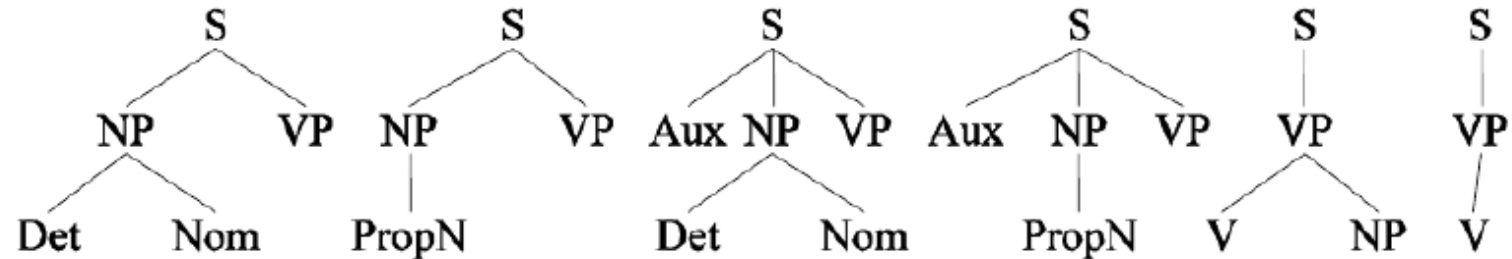
- The Start symbol

S

- Can derive 3 rules as follows:



- Each non-terminal can derive additional rules



- Only the last two trees can derive the word “book” as first in the input

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid ight \mid meal \mid money$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid prefer$
$NP \rightarrow Pronoun$	$Pronoun \rightarrow I \mid she \mid me$
$NP \rightarrow Proper-Noun$	$Proper-Noun \rightarrow Houston \mid TWA$
$NP \rightarrow Det Nominal$	$Aux \rightarrow does$
$Nominal \rightarrow Noun$	$Preposition \rightarrow from \mid to \mid on \mid near \mid through$
$Nominal \rightarrow Nominal Noun$	
$Nominal \rightarrow Nominal PP$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	
$VP \rightarrow Verb NP PP$	
$VP \rightarrow Verb PP$	
$VP \rightarrow VP PP$	
$PP \rightarrow Preposition NP$	

Bottom-up Parser

- Data-driven
- Looks at words in input string first, checks / assigns their category(ies), and tries to combine them into acceptable structures in the grammar
- Involves scanning the derivation so far for sub-strings which match the right-hand-side of grammar / production rules and using the rule that would show their derivation from the non-terminal symbol of that rule.

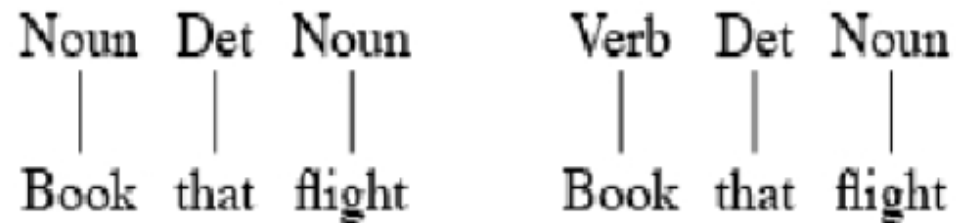
Bottom-up Derivation

- Starts with input text

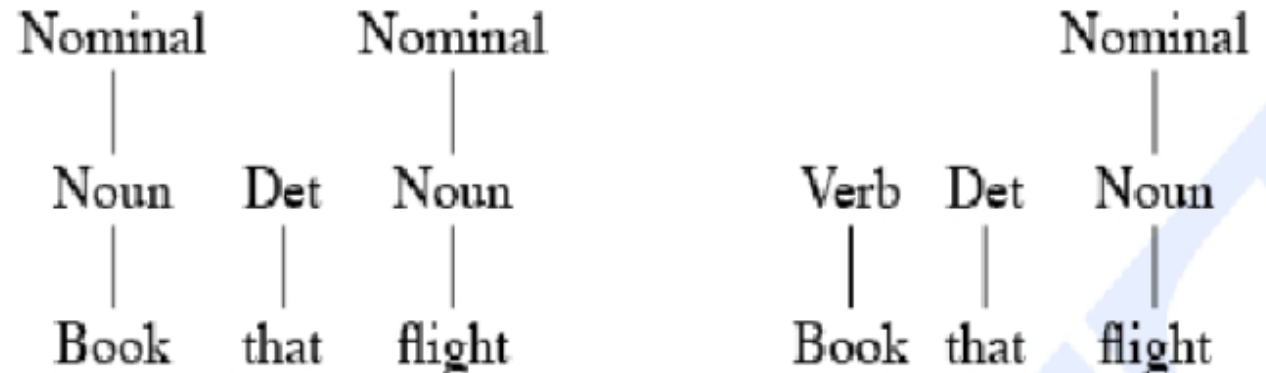
$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid ight \mid meal \mid money$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid prefer$
$NP \rightarrow Pronoun$	$Pronoun \rightarrow I \mid she \mid me$
$NP \rightarrow Proper-Noun$	$Proper-Noun \rightarrow Houston \mid TWA$
$NP \rightarrow Det Nominal$	$Aux \rightarrow does$
$Nominal \rightarrow Noun$	$Preposition \rightarrow from \mid to \mid on \mid near \mid through$
$Nominal \rightarrow Nominal Noun$	
$Nominal \rightarrow Nominal PP$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	
$VP \rightarrow Verb NP PP$	
$VP \rightarrow Verb PP$	
$VP \rightarrow VP PP$	
$PP \rightarrow Preposition NP$	

Book that flight

- derive the text from rules, in this case, two possible lexical rules

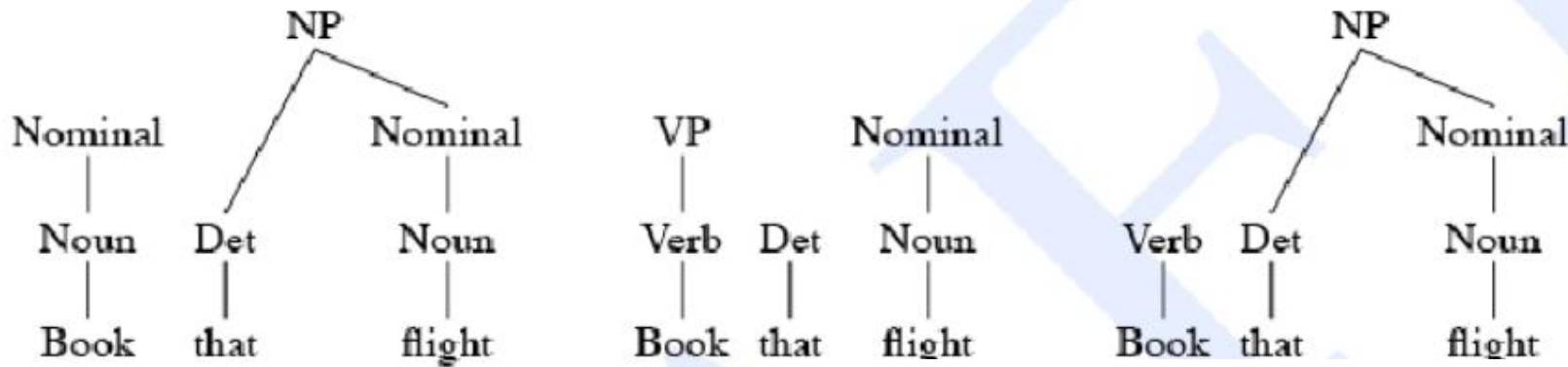


- Each of those can be derived from nonterminals

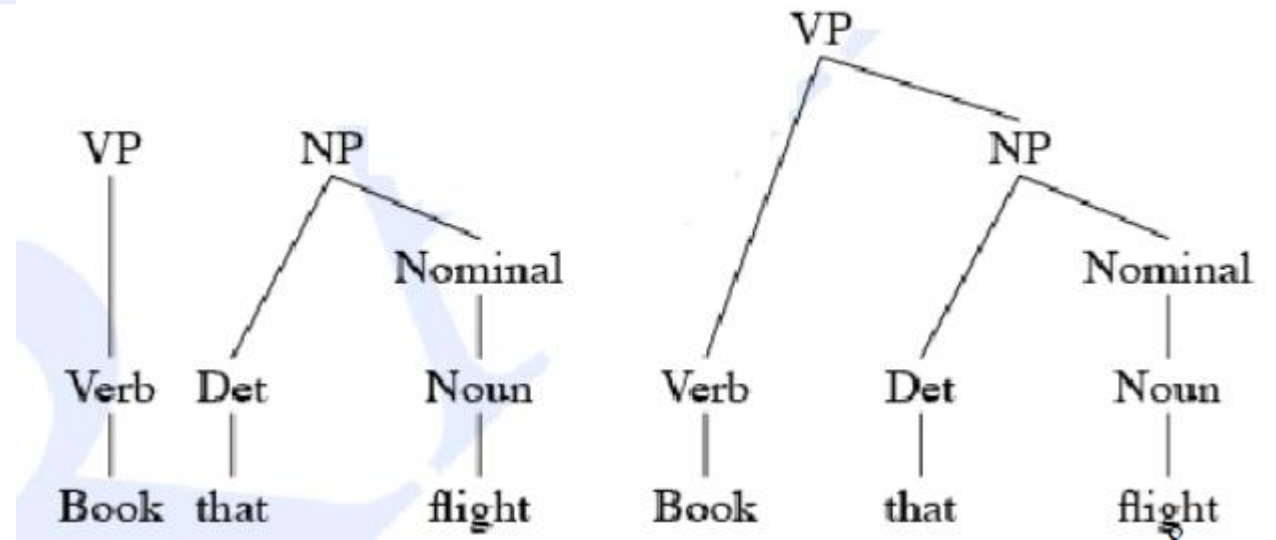


Bottom-up Derivation

- Only the rightmost tree can continue the derivation here:



- And only one succeeds: $S \rightarrow VP$



Bottom-up Parsing

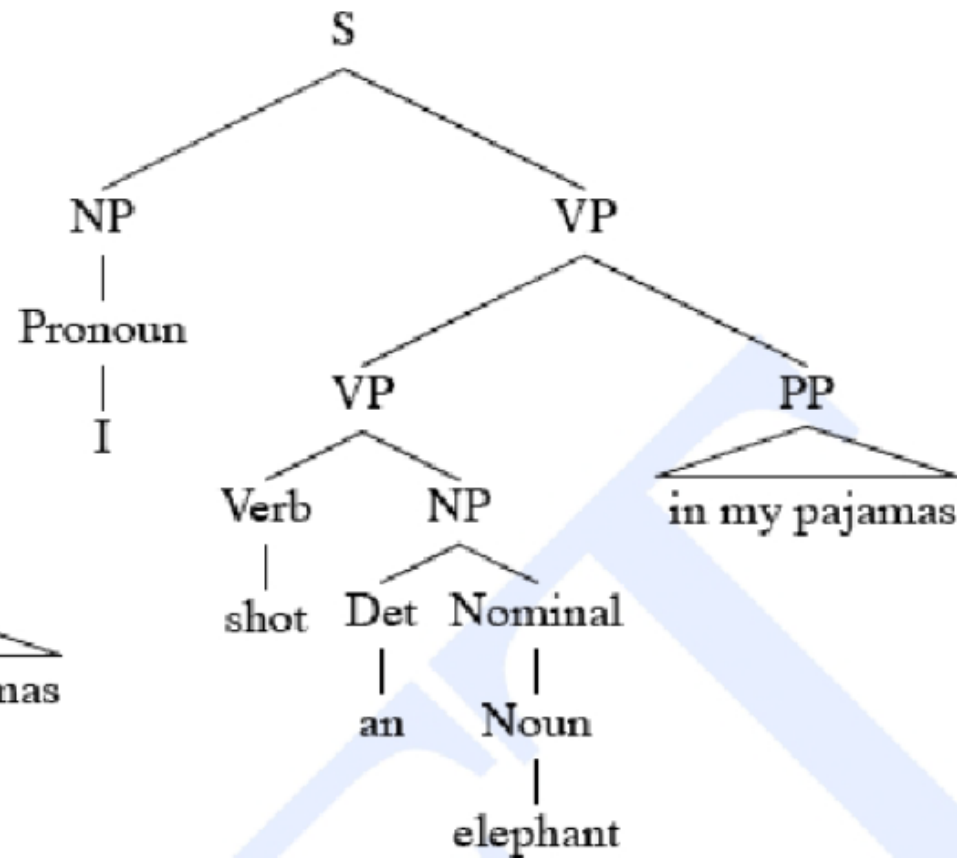
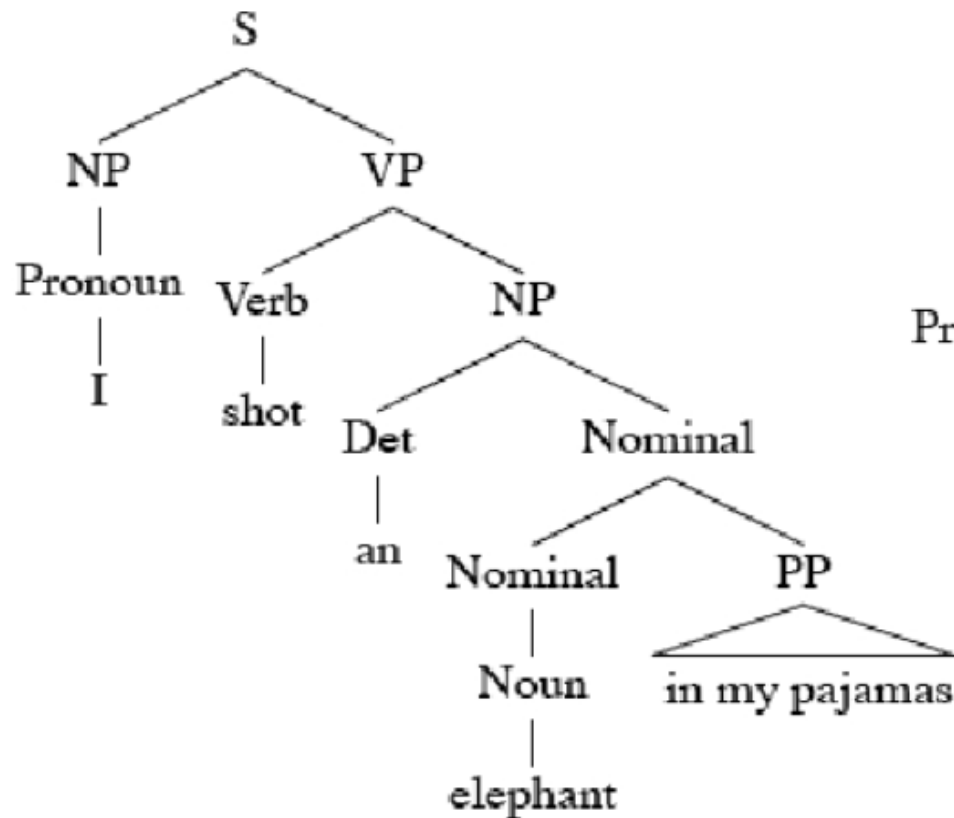
- Algorithm called shift/reduce parsing
 - Scans the input from left to right and keeps a “stack” of the partial parse tree so far
 - The shift operation looks at the next input and shifts it onto the stack
 - The reduce operation looks at N symbols on the stack and if they match the RHS of a grammar rule, reduces the stack by replacing those symbols with the non-terminal
- Also must either incorporate back-tracking or must keep multiple possible parses

Parsing issues

- Top-down
 - Only searches for trees that can be answers (i.e. S's)
 - But also suggests trees that are not consistent with any of the words
- Bottom-up
 - Only forms trees consistent with the words
 - But suggest trees that make no sense globally
- Note that in the previous example, there was lexical ambiguity between “book” being a verb or a noun that was resolved at the end of the parse
- But examples with structural ambiguity will not be resolved, resulting in more than one possible derivation

Structural Ambiguity

- *One morning I shot an elephant in my pajamas. How he got into my pajamas I don't know.* Groucho Marx, *Animal Crackers*, 1930.



Questions

- | What kind of ambiguity is there in the following sentences? Justify your answers by drawing parse trees or using any other method.
 - (i) I invited the person with the microphone|
 - (ii) I saw him at the bank

Parsing Algorithms

- The simple parsers that we have seen are exponential in time (recursive descent with back tracking) and (shift reduce with back-tracking)
- Avoid back-tracking and re-doing sub-trees
 - Recall that the backtracking recursive descent expanded some sub-trees multiple times
- Use forms of dynamic programming to search for good parse trees
 - Attempt to perform exponential process in polynomial time

Recursive descent parsing

- The parse tree is constructed
 - Top-down
 - By reading the string from left to right
 - Try the production in order.

Recursive descent parsing

- Example 1

$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

Token class stream:
(int)

Try E production **in order**,

$E \rightarrow T$ then

$E \rightarrow T + E$

Recursive descent parsing

- Example

$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

Token class stream

(int)

Try E production **in order**,

E
↓
T
↓
int

Does not match with the terminal (- Backtrack

Recursive descent parsing

- Example

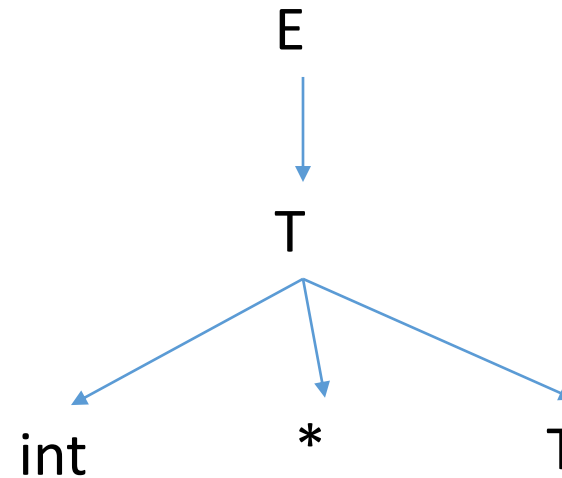
$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

Token class stream
(int)

Try E production in
order,

Backtrack to a node where
another alternative is available



Does not match with the
terminal (- Backtrack

Recursive descent parsing

- Example

$E \rightarrow T \mid T + E$

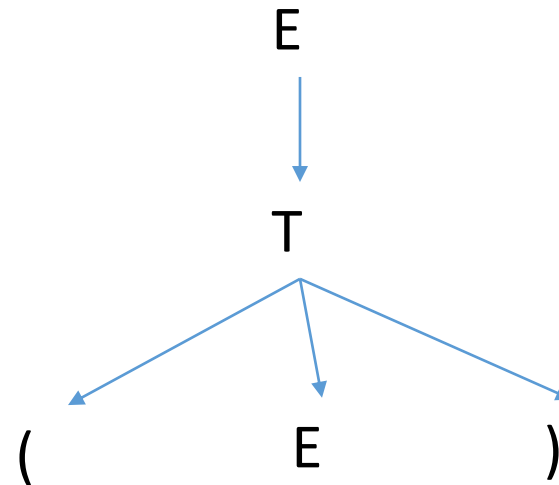
$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

Token class stream

(int)

Try E production **in order**,

Backtrack to a node where another alternative is available



Match with the terminal (- Move to the next token and the next node in the parse tree

Recursive descent parsing

- Example

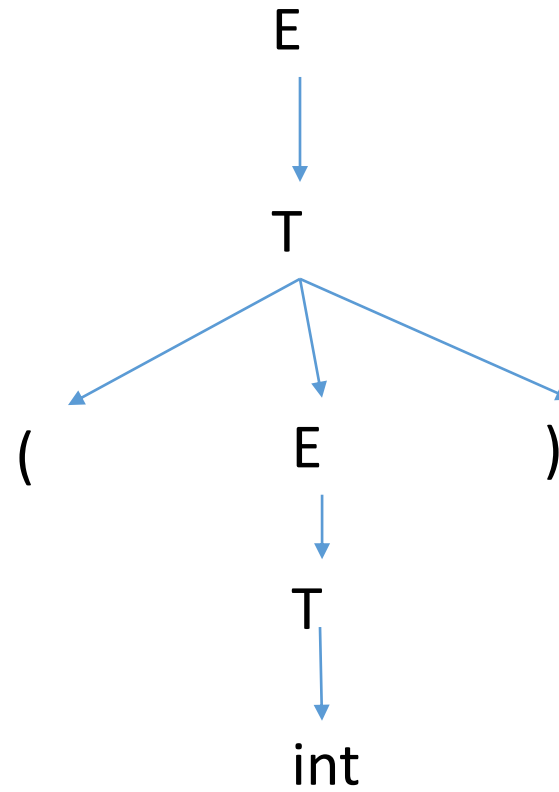
$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

Token class stream

(int)

End of input – Accept
the string



Need for Treebanks

- Before you can parse you need a grammar.
- So where do grammars come from?
 - Grammar Engineering
 - Hand-crafted decades-long efforts by humans to write grammars (typically in some particular grammar formalism of interest to the linguists developing the grammar).
 - TreeBanks
 - Semi-automatically generated sets of parse trees for the sentences in some corpus. Typically in a generic lowest common denominator formalism (of no particular interest to any modern linguist, but representing phrases of text in actual use).

The rise of annotated data

- Starting off, building a tree-bank seems a lot slower and less useful than building a grammar
- But a treebank gives us many things
 - Reusability of the labor
 - Many parsers, POS taggers, etc.
 - Valuable resource for linguistics
 - Broad coverage
 - Frequencies and distributional information
 - A way to evaluate systems

Penn Treebank

[Marcus et al. 1993, *Computational Linguistics*]

```
( (S
  (NP-SBJ (DT The) (NN move))
  (VP (VBD followed)
    (NP
      (NP (DT a) (NN round))
      (PP (IN of)
        (NP
          (NP (JJ similar) (NNS increases))
          (PP (IN by)
            (NP (JJ other) (NNS lenders))))
          (PP (IN against)
            (NP (NNPARizona) (JJ real) (NN estate) (NNS loans)))))))
    (, ,)
    (S-ADV
      (NP-SBJ (-NONE- *))
      (VP (VBG reflecting)
        (NP
          (NP (DT a) (VBG continuing) (NN decline))
          (PP-LOC (IN in)
            (NP (DT that) (NN market)))))))
  (. .)))
```


Getting grammar from a treebank

- Given an annotated sentence.

(11.10) [NP Shearson's] [JJ easy-to-film], [JJ black-and-white] “[SBAR Where We Stand]” [NNS commercials]

- We can make a grammar rule:

$$\text{NP} \rightarrow \text{NP JJ , JJ `` SBAR `` NNS}$$

- And we will make rules for sub-trees as well

Sample Rules for Noun Phrases

NP → DT JJ NNS
NP → DT JJ NN NN
NP → DT JJ JJ NN
NP → DT JJ CD NNS
NP → RB DT JJ NN NN
NP → RB DT JJ JJ NNS
NP → DT JJ JJ NNP NNS
NP → DT NNP NNP NNP NNP JJ NN
NP → DT JJ NNP CC JJ JJ NN NNS
NP → RB DT JJS NN NN SBAR
NP → DT VBG JJ NNP NNP CC NNP
NP → DT JJ NNS , NNS CC NN NNS NN
NP → DT JJ JJ VBG NN NNP NNP FW NNP
NP → NP JJ , JJ `` SBAR `` NNS

TreeBank Grammars

- Reading off the grammar...
- The grammar is the set of rules (local sub-trees) that occur in the annotated corpus
- They tend to avoid recursion (and elegance and parsimony)
 - i.e. they tend to the flat and redundant
- Penn TreeBank (III) has about 17500 grammar rules under this definition.
- But the main use of the Treebank is to provide the probabilities to inform the statistical parsers, and the grammar does not actually have to be generated.
- The grammar hovers behind the Treebank; it is in the minds of the human annotators (and in the annotation manual!)

Probabilistic Context-Free Grammars

- By way of introduction to statistical parsers, we first introduce the idea of associating probabilities with grammar rewrite rules.
 - Attach probabilities to grammar rules
 - The expansions for a given non-terminal sum to 1

VP → Verb .55

VP → Verb NP .40

VP → Verb NP NP .05

Probabilistic Context-Free Grammars

S	$\Rightarrow$	NP	VP	1.0
VP	$\Rightarrow$	Vi		0.4
VP	$\Rightarrow$	Vt	NP	0.4
VP	$\Rightarrow$	VP	PP	0.2
NP	$\Rightarrow$	DT	NN	0.3
NP	$\Rightarrow$	NP	PP	0.7
PP	$\Rightarrow$	P	NP	1.0

Vi	$\Rightarrow$	sleeps	1.0
Vt	$\Rightarrow$	saw	1.0
NN	$\Rightarrow$	man	0.7
NN	$\Rightarrow$	woman	0.2
NN	$\Rightarrow$	telescope	0.1
DT	$\Rightarrow$	the	1.0
IN	$\Rightarrow$	with	0.5
IN	$\Rightarrow$	in	0.5

- Probability of a tree t with rules

$$\alpha_1 \rightarrow \beta_1, \alpha_2 \rightarrow \beta_2, \dots, \alpha_n \rightarrow \beta_n$$

is $p(t) = \prod_{i=1}^n q(\alpha_i \rightarrow \beta_i)$ where $q(\alpha \rightarrow \beta)$ is the probability for rule $\alpha \rightarrow \beta$.

Calculating probabilities

S	⇒	NP	VP	1.0
VP	⇒	Vi		0.4
VP	⇒	Vt	NP	0.4
VP	⇒	VP	PP	0.2
NP	⇒	DT	NN	0.3
NP	⇒	NP	PP	0.7
PP	⇒	P	NP	1.0

Vi	⇒	sleeps	1.0
Vt	⇒	saw	1.0
NN	⇒	man	0.7
NN	⇒	woman	0.2
NN	⇒	telescope	0.1
DT	⇒	the	1.0
IN	⇒	with	0.5
IN	⇒	in	0.5

- The man sleeps
- The woman saw the telescope

Calculating probabilities

S	⇒	NP	VP	1.0
VP	⇒	Vi		0.4
VP	⇒	Vt	NP	0.4
VP	⇒	VP	PP	0.2
NP	⇒	DT	NN	0.3
NP	⇒	NP	PP	0.7
PP	⇒	P	NP	1.0

Vi	⇒	sleeps	1.0
Vt	⇒	saw	1.0
NN	⇒	man	0.7
NN	⇒	woman	0.2
NN	⇒	telescope	0.1
DT	⇒	the	1.0
IN	⇒	with	0.5
IN	⇒	in	0.5

- The man sleeps = $1.0 \times 0.3 \times 1.0 \times 0.7 \times 0.4 \times 1.0 = 0.084$
- The woman saw the telescope = $1.0 \times 0.3 \times 1.0 \times 0.2 \times 0.4 \times 1.0 \times 0.3 \times 1.0 \times 0.1 = 0.00072$

Getting the probabilities

- From a treebank of annotated data, get the probabilities that any non-terminal symbol is rewritten with a particular rule
 - So for example, to get the probability for a particular VP rule just count all the times the rule is used and divide by the number of VPs overall.
- The parsing task is to generate the parse tree with the highest probability (or the top n parse trees)
- The probability of a parse tree is the product of the probabilities of the rules used in the derivation

$$P(T, S) = \prod_{node \in T} P(rule(n))$$

Problems with PCFG Parsing

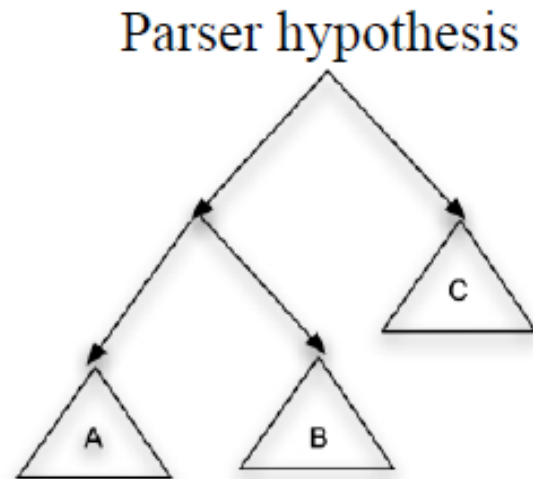
- But this typical approach always just picks the most likely rule in the derivation
 - For example, if it is more likely that a prepositional phrase attaches to the noun phrase that it follows instead of the verb, then the probabilistic parser will always attach prepositional phrases to the closest noun
- The probability model we are using is only based on the rules in the derivation...
 - Doesn't use the words in any real way
 - Doesn't take into account where in the derivation a rule is used
 - E.g. the parent of the non-terminal of the derivation
- Doesn't really work
 - Most probable parse isn't usually the right one (the one in the treebank test set).

Evaluation

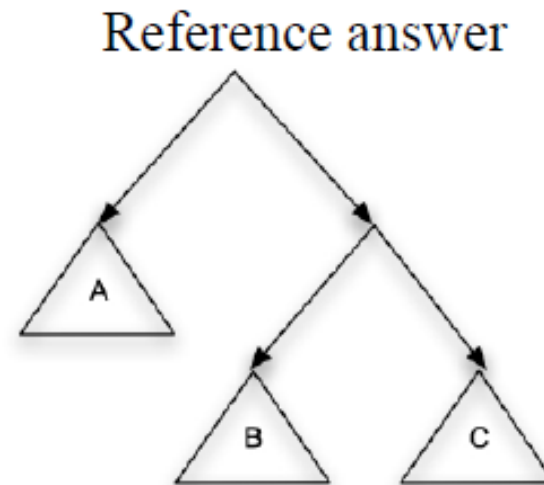
- Given that it is difficult/ambiguous to produce the entire correct tree, look at how much of content of the trees are correctly produced
 - Evaluation measures based on the correct number of constituents (or sub-trees) in the system compared to the reference (gold standard)
- Precision
 - What fraction of the sub-trees in our parse matched corresponding sub-trees in the reference answer
 - How much of what we're producing is right?
- Recall
 - What fraction of the sub-trees in the reference answer did we actually get?
 - How much of what we should have gotten did we get?
- F-measure combines precision and recall to give an overall

Evaluation

- An additional evaluation measure that is often reported is that of Crossing Brackets errors, in which the subtrees are equal, but they are put together in a different order.



$((AB) C)$



$(A(B C))$

Available Parsers

- Among the family of lexicalized statistical parsers are the wellknown Collins parser (Michael Collins 1996, 1999) and the Charniak parser (1997)
 - both are publicly available and widely used in NLP, for non-commercial purposes.
- The Charniak series of parsers is still under development, by Eugene Charniak and his group; it produces N-best parse trees.
 - Its evaluation is currently the best on the Penn Treebank at about 91% F measure.
- Another top performing parser, originally by Dan Klein and Christopher Manning, is available from the Stanford NLP group
 - combines “separate PCFG phrase structure and lexical dependency experts”.
 - Demo at: <http://nlp.stanford.edu:8080/parser/>

Available Parsers

- The CCG parsers are available from their open source page
 - <http://groups.inf.ed.ac.uk/ccg/software.html>
- Parsers are also available through the OpenNLP project, with the OpenNLP API:
 - <http://opennlp.sourceforge.net/>

Training the Parser

- How does the parser know which operation, and arc label, to apply?
 - It learns these from the annotated corpus.
 - For English, dependency grammar relations are derived from Penn Treebank
 - Then a collection of examples is extracted from the text to train the parser
 - Each example consists of a set of features representing the state of the parser, including the next word, previous word, POS tags, etc.
 - A machine learning algorithm is applied to learn a classifier, which can assign a parsing operation to every parsing state

Questions!

- What among following is an/are example(s) of syntactic ambiguity?
 - The man went to the bank.
 - He saw the man in his sarong.
 - All basketball players are tall.
 - Colourless green ideas sleep furiously.

Questions!

- What among following is an/are example(s) of syntactic ambiguity?
 - The man went to the bank.
 - He saw the man in his sarong.
 - All basketball players are tall.
 - Colourless green ideas sleep furiously.